

STM32F103C8T6流水灯实验报告

一、实验目的

- 掌握STM32F103 GPIO寄存器底层操作，理解GPIO相关寄存器地址与配置参数。
- 使用GPIOA、GPIOB端口引脚外接3个LED，实现3个LED轮流闪烁，间隔1秒。
- 理解APB2外设时钟、GPIO端口配置寄存器、BSRR置位复位寄存器工作原理。
- 拓展学习：使用最小系统板自带PC13板载LED，将PC13加入流水灯程序。

二、实验器材

STM32F103C8T6最小系统核心板、面包板、LED发光二极管3只、220Ω限流电阻、杜邦线、Keil MDK开发环境。

三、实验原理

STM32F103系列GPIO挂载在APB2总线上，使用GPIO端口前必须开启对应外设时钟。

GPIO低引脚（0~7）使用CRL寄存器配置，可设置端口输出模式、输出速度。

BSRR寄存器：低16位写1，对应引脚置高电平；高16位写1，对应引脚清零，不会产生读改写冲突，适合控制引脚电平。

GPIO寄存器地址与参数

寄存器名称	起始地址	功能说明
RCC_APB2ENR	0x40021018	APB2外设时钟使能寄存器
GPIOA_CRL	0x40010800	GPIOA引脚0~7配置寄存器
GPIOB_CRL	0x40010C00	GPIOB引脚0~7配置寄存器
GPIOA_BSRR	0x40010810	GPIOA置位/复位寄存器
GPIOB_BSRR	0x40010C10	GPIOB置位/复位寄存器

本次选用引脚：PA0、PB0、PB1，全部属于0~7号引脚，使用CRL寄存器配置为推挽输出，最大速度50MHz。

四、硬件接线

- LED1：220Ω限流电阻串联，一端接PA0，LED负极接GND。
- LED2：220Ω限流电阻串联，一端接PB0，LED负极接GND。
- LED3：220Ω限流电阻串联，一端接PB1，LED负极接GND。

五、程序设计思路

- 开启RCC_APB2ENR寄存器中GPIOA、GPIOB外设时钟。
- 配置GPIOA_CRL，将PA0设置为推挽输出，50MHz。
- 配置GPIOB_CRL，将PB0、PB1设置为推挽输出，50MHz。

4. 编写软件延时函数，实现约1秒延时。
5. 在主循环中，依次点亮一个LED，延时1秒，熄灭当前LED，切换下一个LED，循环往复。
6. 使用BSRR寄存器控制引脚高低电平。

六、寄存器方式C语言代码（带详细注释）

```
``c
// 寄存器地址宏定义
#define RCC_APB2ENR (*(volatile unsigned int *)0x40021018)
#define GPIOA_CRL (*(volatile unsigned int *)0x40010800)
#define GPIOB_CRL (*(volatile unsigned int *)0x40010C00)
#define GPIOA_BSRR (*(volatile unsigned int *)0x40010810)
#define GPIOB_BSRR (*(volatile unsigned int *)0x40010C10)

/**
 * @brief 软件毫秒延时函数
 * @param ms 延时毫秒数
 */
void Delay_ms(uint32_t ms)
{
    uint32_t i,j;
    for(i=0; i<ms; i++)
        for(j=0; j<7200; j++);
}

/**
 * @brief LED初始化，配置PA0 PB0 PB1为推挽输出
 */
void LED_Init(void)
{
    // 开启GPIOA、GPIOB APB2时钟
    RCC_APB2ENR |= (1 << 2) | (1 << 3);

    // PA0: 推挽输出 50MHz, CRL bit0~3
    GPIOA_CRL &= ~(0x0F << 0);
    GPIOA_CRL |= (0x03 << 0);

    // PB0 PB1: 推挽输出 50MHz
    GPIOB_CRL &= ~((0x0F << 0) | (0x0F << 4));
    GPIOB_CRL |= ((0x03 << 0) | (0x03 << 4));

    // 所有LED初始熄灭
    GPIOA_BSRR = (1 << (0 + 16));
    GPIOB_BSRR = (1 << (0 + 16)) | (1 << (1 + 16));
}

int main(void)
{
    LED_Init();
    while(1)
    {
        // PA0点亮，延时1s
    }
}
```

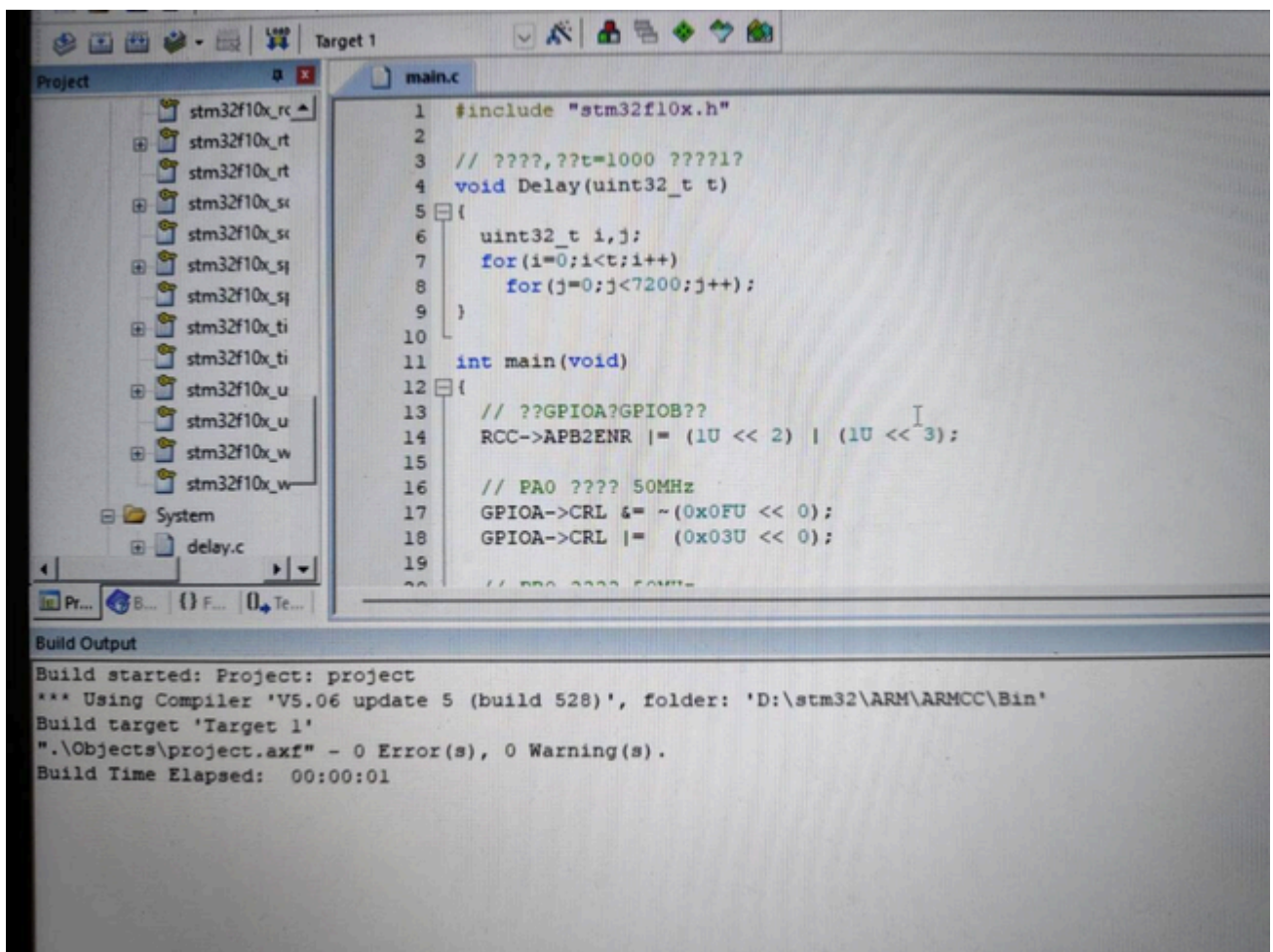
```

GPIOA_BSRR = 1 << 0;
Delay_ms(1000);
GPIOA_BSRR = 1 << (0 + 16);

// PB0点亮, 延时1s
GPIOB_BSRR = 1 << 0;
Delay_ms(1000);
GPIOB_BSRR = 1 << (0 + 16);

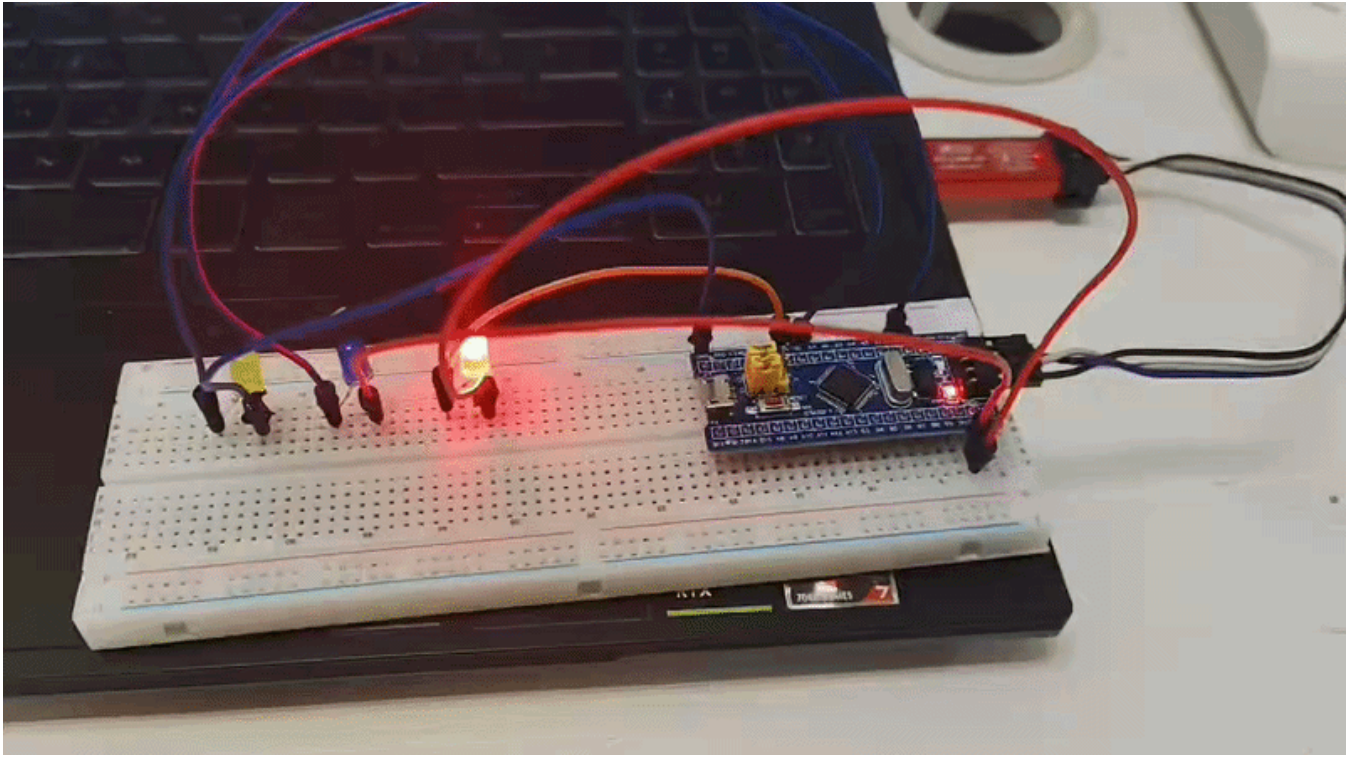
// PB1点亮, 延时1s
GPIOB_BSRR = 1 << 1;
Delay_ms(1000);
GPIOB_BSRR = 1 << (1 + 16);
}
}

```



七、实验现象

程序烧录完成后, 3个LED按照PA0 → PB0 → PB1顺序轮流点亮, 每个LED点亮保持1秒, 熄灭后切换下一个LED, 不断循环。



八、拓展任务

将板载PC13加入流水灯 最小系统板自带PC13LED，查阅原理图，PC13为低电平点亮。在原有代码基础上增加GPIOC寄存器配置，修改程序实现4灯流水。

```
C

#define GPIOC_CRH    (*(volatile unsigned int *)0x40011004)
#define GPIOC_BSRR   (*(volatile unsigned int *)0x40011010)

void LED_Init(void)
{
    RCC_APB2ENR |= (1 << 2) | (1 << 3) | (1 << 4);
    // PA0 PB0 PB1配置同上
    GPIOC_CRH &= ~(0x0F << 20);
    GPIOC_CRH |= (0x03 << 20);
    GPIOC_BSRR = 1 << 13;
}

int main(void)
{
    LED_Init();
    while(1)
    {
        GPIOA_BSRR = 1 << 0;
        Delay_ms(1000);
        GPIOA_BSRR = 1 << (0 + 16);

        GPIOB_BSRR = 1 << 0;
        Delay_ms(1000);
```

```
        GPIOB_BSRR = 1 << (0 + 16);

        GPIOB_BSRR = 1 << 1;
        Delay_ms(1000);
        GPIOB_BSRR = 1 << (1 + 16);

        GPIOC_BSRR = 1 << (13 + 16);
        Delay_ms(1000);
        GPIOC_BSRR = 1 << 13;
    }
}
```

九、实验总结

1. 使用寄存器直接操作方式可以直接配置GPIO硬件，需要掌握寄存器地址和每一位参数含义
2. 使用GPIO端口前，必须开启对应APB2外设时钟，否则GPIO无法正常工作。
3. BSRR寄存器可以单独置位、清零引脚，不会干扰其他引脚，是控制GPIO输出的推荐方式。
4. 软件延时依靠循环空等待实现，适合简单流水灯实验，延时精度一般。
5. 拓展部分验证了板载PC13LED，PC13为低电平点亮，电平逻辑和外接LED相反。